

Avaliação Comparativa de Algoritmos de Filtragem Colaborativa no Dataset MovieLens 100k

Júlia da Silva Borges
Universidade Federal Rural do Rio de Janeiro
Nova Iguaçu, RJ, Brasil
brgsju@ufrj.br

Charlie Prince de Souza Andrade
Universidade Federal Rural do Rio de Janeiro
Nova Iguaçu, RJ, Brasil
charliepsa04@ufrj.br

1 Introdução

O crescimento exponencial do comércio eletrônico e das plataformas de *streaming* trouxe consigo um fenômeno conhecido como sobrecarga de informação (*information overload*): os usuários se deparam com catálogos vastos de produtos e conteúdos, o que dificulta a tomada de decisão de consumo. Nesse contexto, os Sistemas de Recomendação (SR) emergem como ferramentas fundamentais para filtrar e personalizar a experiência do usuário, auxiliando-o a encontrar itens relevantes dentro de um espaço de busca extenso [1].

A filtragem colaborativa (*Collaborative Filtering*) é uma das abordagens mais amplamente estudadas e adotadas em SR. Seu princípio fundamental é o de que usuários com histórico de avaliações semelhante tendem a ter preferências similares no futuro. A CF pode ser dividida em duas grandes famílias: as abordagens **baseadas em memória** (*memory-based*), que operam diretamente sobre a matriz de avaliações para identificar vizinhos similares; e as abordagens **baseadas em modelo** (*model-based*), que aprendem uma representação latente dos usuários e itens a partir dos dados de treino [4].

A competição Netflix Prize, realizada entre 2006 e 2009, impulsionou de forma significativa o desenvolvimento de algoritmos de CF, evidenciando o potencial das técnicas de fatoração de matrizes, em especial o SVD, para aprimorar substancialmente a acurácia da predição de avaliações [2]. Desde então, variações como o SVD++ passaram a incorporar informações implícitas de comportamento do usuário, obtendo resultados ainda superiores.

Este trabalho tem como objetivo implementar, hiperparametrizar e avaliar comparativamente algoritmos de CF sobre o *dataset* MovieLens 100k, um *benchmark* consolidado na comunidade de SR. Especificamente, são implementados:

- **KNN User-Based e Item-Based** — algoritmos baseados em memória, com similaridade do cosseno sobre a matriz centralizada na média (*mean-centered*);
- **SVD Regularizado** — algoritmo baseado em modelo por fatoração de matrizes com descida de gradiente estocástico;
- **SVD++** — extensão do SVD que incorpora *feedback* implícito dos itens avaliados pelo usuário.

O restante do trabalho está organizado da seguinte forma: a Seção 2 descreve o *dataset*, a metodologia de avaliação e os algoritmos implementados; a Seção 3 apresenta e discute os resultados obtidos; e a Seção 4 sintetiza as conclusões e aponta direções futuras.

2 Experimentos

2.1 Base de Dados

O *dataset* utilizado é o **MovieLens 100k** [5], disponibilizado pelo GroupLens Research Lab da Universidade de Minnesota. Trata-se de

um *benchmark* amplamente utilizado na literatura de SR, contendo avaliações de filmes coletadas entre setembro de 1997 e abril de 1998. A Tabela 1 resume suas principais características.

Tabela 1: Características do Dataset MovieLens 100k

Característica	Valor
Total de avaliações	100.000
Total de usuários	943
Total de filmes	1.682
Escala de avaliações	1 a 5 (inteiros)
Avaliação média	3,53
Esparsidade da matriz	93,70%
Usuário mais ativo	737 avaliações
Usuário mediano	65 avaliações
Filme mais avaliado	583 avaliações
Proporção M/F dos usuários	71% / 29%
Idade média dos usuários	34,1 anos

A matriz usuário-item possui dimensões 943×1682 com esparsidade de 93,70%, ou seja, apenas 6,30% dos pares (usuário, filme) possuem avaliação registrada. Essa característica é inerente a *datasets* de SR e motiva o uso de técnicas de filtragem colaborativa, capazes de gerar predições mesmo para pares não observados.

A distribuição das notas revela assimetria negativa, com concentração nos valores 4 e 5, padrão típico de *selection bias* em SR: usuários tendem a avaliar filmes que escolheram assistir e, portanto, têm maior probabilidade de apreciar [4]. Os histogramas de avaliações por usuário e por filme evidenciam distribuição de cauda longa (*long-tail*), com a maioria dos usuários realizando poucas avaliações e uma minoria concentrando grande volume, fenômeno que acentua a esparsidade e desafia os algoritmos de CF.

2.2 Pré-processamento: Centralização por Média

Para o KNN, as avaliações foram centralizadas pela média do usuário antes do cálculo de similaridade, técnica conhecida como *mean-centering*. Formalmente, dado r_{ui} como a nota do usuário u ao item i e $\bar{r}_u$ como a média das notas de u , define-se a nota centralizada:

$$\tilde{r}_{ui} = r_{ui} - \bar{r}_u \quad (1)$$

A centralização é necessária porque diferentes usuários utilizam a escala de notas de forma distinta: um usuário que avalia tudo entre 3 e 5 e outro que avalia entre 1 e 3 podem ter preferências similares, mas o cosseno sem centralização interpretaria seus perfis como dissimilares. Ao remover o viés individual de escala, a similaridade cosseno passa a capturar *padrões relativos* de preferência, não

Análise Exploratória do Dataset MovieLens 100k

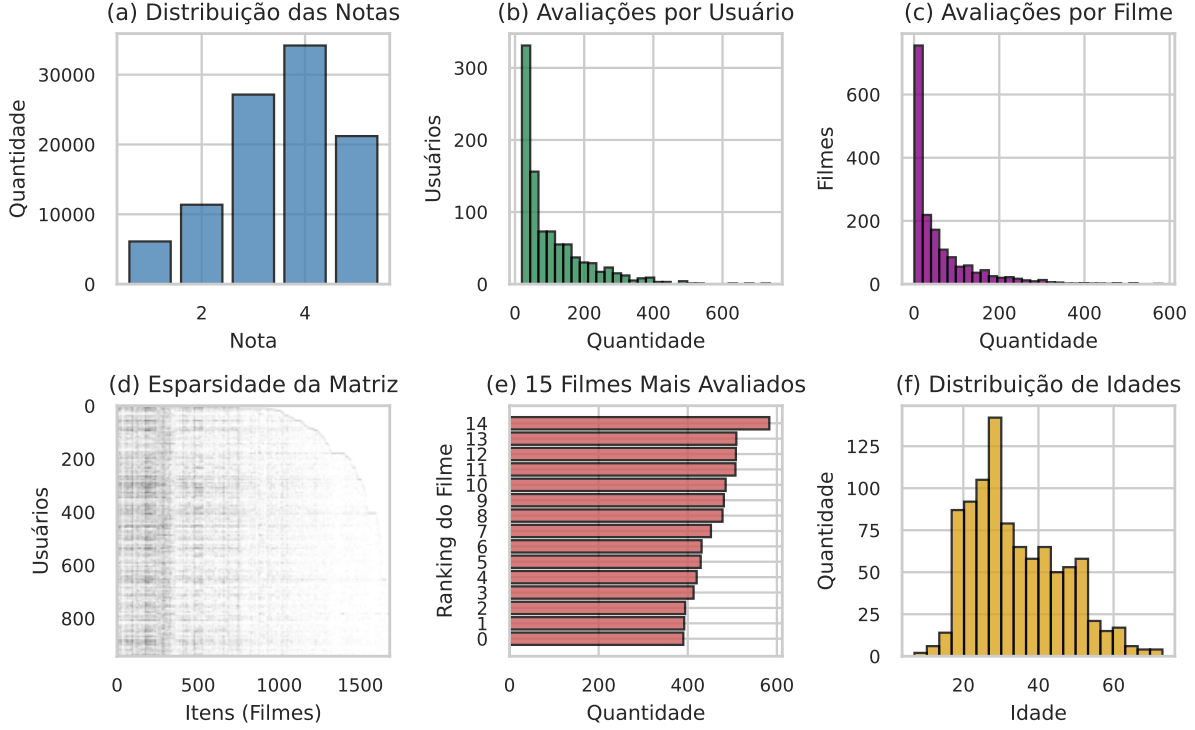


Figura 1: Análise exploratória do MovieLens 100k: (a) distribuição das notas; (b) avaliações por usuário; (c) avaliações por filme; (d) esparsidade da matriz usuário-item; (e) 15 filmes mais avaliados; (f) distribuição de idades.

magnitudes absolutas [4]. Entradas não avaliadas são preenchidas com zero após a centralização, o que é equivalente a assumir que a preferência relativa do usuário por itens não avaliados é neutra.

2.3 Metodologia de Avaliação

2.3.1 Validação Cruzada K-Fold. A metodologia adotada é a **validação cruzada K-Fold** com $K = 5$, implementada do zero sem uso de bibliotecas externas de aprendizado de máquina. O procedimento embaralha aleatoriamente o conjunto de avaliações (semente *random_state* = 42 para reprodutibilidade), divide em $K = 5$ partições de tamanho aproximadamente igual (≈ 20.000 avaliações cada) e, para cada fold k , usa o fold k como teste e os $K - 1$ restantes como treino.

A escolha do K-Fold em detrimento do *hold-out* simples é motivada pela alta esparsidade do dataset (93,70%): com um único split, parte dos usuários ou filmes poderia não aparecer no treino, introduzindo viés nas estimativas. O K-Fold garante que **todas as avaliações participem do teste exatamente uma vez**, reduzindo a variância da estimativa de desempenho [4]. O valor $K = 5$ representa o equilíbrio padrão da literatura entre o *bias-variance tradeoff* e o custo computacional [4].

Para a busca de hiperparâmetros, utilizaram-se apenas os 3 primeiros folds (*inner loop*), reduzindo o custo computacional sem

comprometer a identificação da região ótima. A avaliação final de cada modelo foi realizada com os 5 folds completos (*outer loop*).

2.3.2 Métricas de Avaliação. Duas métricas complementares são utilizadas para avaliar a qualidade da predição de notas:

- **MAE** (*Mean Absolute Error*):

$$\text{MAE} = \frac{1}{|\hat{R}|} \sum_{(u,i) \in \hat{R}} |r_{ui} - \hat{r}_{ui}| \quad (2)$$

Penaliza erros linearmente, sendo de interpretação direta na escala original das avaliações (1 a 5).

- **RMSE** (*Root Mean Squared Error*):

$$\text{RMSE} = \sqrt{\frac{1}{|\hat{R}|} \sum_{(u,i) \in \hat{R}} (r_{ui} - \hat{r}_{ui})^2} \quad (3)$$

Penaliza erros grandes mais severamente por elevar as diferenças ao quadrado. É a métrica padrão da literatura desde o Netflix Prize [2], permitindo comparação com trabalhos publicados.

A comparação conjunta de MAE e RMSE informa sobre a distribuição dos erros: quando $\text{RMSE} \gg \text{MAE}$, há presença de erros pontuais grandes [4].

2.4 Algoritmos Implementados

2.4.1 Baseline — Média Global. Como limite inferior de comparação, implementa-se um preditor que sempre retorna a média global das avaliações do conjunto de treino. Todo algoritmo útil deve superar esse *baseline*. Os resultados estão na Tabela 2.

Tabela 2: Resultados do Baseline (Média Global) — K-Fold ($K = 5$)

Fold	MAE	RMSE
1	0,9444	1,1239
2	0,9414	1,1215
3	0,9414	1,1232
4	0,9456	1,1266
5	0,9507	1,1331
Média	0,9447	1,1257
± Desvio	0,0038	0,0046

2.4.2 KNN — Filtragem Colaborativa Baseada em Memória. O KNN é um algoritmo *lazy*: não há uma fase de treino explícita no sentido paramétrico, o modelo armazena os dados e realiza a predição consultando os vizinhos mais similares no momento da inferência. Foram implementadas duas variantes:

User-Based: a predição da nota do usuário u para o item i é dada pela soma ponderada das notas centralizadas dos K vizinhos mais similares:

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N_K(u)} \text{sim}(u, v) \cdot \tilde{r}_{vi}}{\sum_{v \in N_K(u)} |\text{sim}(u, v)|} \quad (4)$$

onde $N_K(u)$ é o conjunto dos K vizinhos mais similares de u que avaliaram i , e $\text{sim}(u, v)$ é a similaridade cosseno entre os perfis centralizados.

Item-Based: análogo ao User-Based, mas a similaridade é calculada entre *itens* em vez de usuários [6]. Embora o MovieLens 100k possua mais itens do que usuários (1.682 filmes vs. 943 usuários), os perfis dos itens tendem a ser mais estáticos e objetivos (gênero, temática) do que as preferências voláteis dos usuários. Isso torna a similaridade item-item historicamente mais estável e confiável em predições [6].

Similaridade Cosseno: a similaridade entre dois usuários u e v é calculada como:

$$\text{sim}(u, v) = \frac{\tilde{\mathbf{r}}_u \cdot \tilde{\mathbf{r}}_v}{\|\tilde{\mathbf{r}}_u\| \cdot \|\tilde{\mathbf{r}}_v\|} \quad (5)$$

A matriz de similaridade completa (943×943 para User-Based e 1682×1682 para Item-Based) é pré-computada uma única vez por fold via multiplicação matricial NumPy ($\mathbf{MM}^T$), evitando o recálculo a cada predição.

Hiperparametrização de K : foram avaliados $K \in \{5, 10, 20, 30, 50\}$ com 3 folds. Esse intervalo foi definido com base em Herlocker et al. [7], que demonstraram empiricamente que valores de K entre 20 e 50 produzem os melhores resultados em datasets similares ao MovieLens. Valores pequenos de K resultam em predições ruidosas por depender de poucos vizinhos, enquanto valores grandes incluem usuários pouco similares, degradando a qualidade. Os resultados da busca estão na Tabela 3.

Tabela 3: Hiperparametrização de K — KNN (3 folds, RMSE médio)

K	User-Based RMSE	Item-Based RMSE
5	0,9707	0,9617
10	0,9428	0,9322
20	0,9314	0,9236
30	0,9293	0,9233
50	0,9295	0,9257
Melhor	$K = 30$	$K = 30$

2.4.3 SVD — Fatoração de Matrizes. O SVD Regularizado [2] é um algoritmo baseado em modelo que fatora a matriz de avaliações em representações latentes de baixa dimensão para usuários e itens. A predição é dada por:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^T \mathbf{q}_i \quad (6)$$

onde μ é a média global, b_u e b_i são vieses de usuário e item, e $\mathbf{p}_u \in \mathbb{R}^f$ e $\mathbf{q}_i \in \mathbb{R}^f$ são os vetores de fatores latentes de dimensão f .

Os parâmetros são aprendidos por **Descida de Gradiente Estocástico (SGD)**: para cada avaliação (u, i, r_{ui}) , calcula-se o erro de predição $e_{ui} = r_{ui} - \hat{r}_{ui}$ e atualizam-se os parâmetros com taxa de aprendizado $\alpha = 0,005$ e regularização $\lambda = 0,02$:

$$\begin{aligned} b_u &\leftarrow b_u + \alpha(e_{ui} - \lambda b_u) \\ b_i &\leftarrow b_i + \alpha(e_{ui} - \lambda b_i) \\ \mathbf{p}_u &\leftarrow \mathbf{p}_u + \alpha(e_{ui} \mathbf{q}_i - \lambda \mathbf{p}_u) \\ \mathbf{q}_i &\leftarrow \mathbf{q}_i + \alpha(e_{ui} \mathbf{p}_u - \lambda \mathbf{q}_i) \end{aligned} \quad (7)$$

O SVD é preferível ao KNN para datasets esparsos pois aprende representações compactas que generalizam melhor para pares não observados, sem depender da existência de vizinhos com avaliações sobrepostas [2].

Hiperparametrização do número de fatores latentes f : foram avaliados $f \in \{5, 10, 20, 50, 100\}$ com 3 folds. Koren et al. [2] reportam que valores entre 20 e 200 fatores produzem bons resultados no MovieLens; valores pequenos levam a *underfitting* (representação insuficiente) e valores excessivos a *overfitting* com dados esparsos. O melhor resultado foi obtido com $f = 100$ (RMSE = 0,9234 na busca), mostrando que o dataset ainda se beneficia de maior capacidade de representação dentro do intervalo testado (Tabela 4).

Tabela 4: Hiperparametrização de f — SVD e SVD++ (3 folds, RMSE médio)

f	SVD RMSE	SVD++ RMSE
5	0,9362	0,9288
10	0,9323	0,9204
20	0,9300	0,9175
50	0,9245	0,9145
100	0,9234	0,9159
Melhor	$f = 100$	$f = 50$

Análise de Hiperparâmetros (Métrica: RMSE)

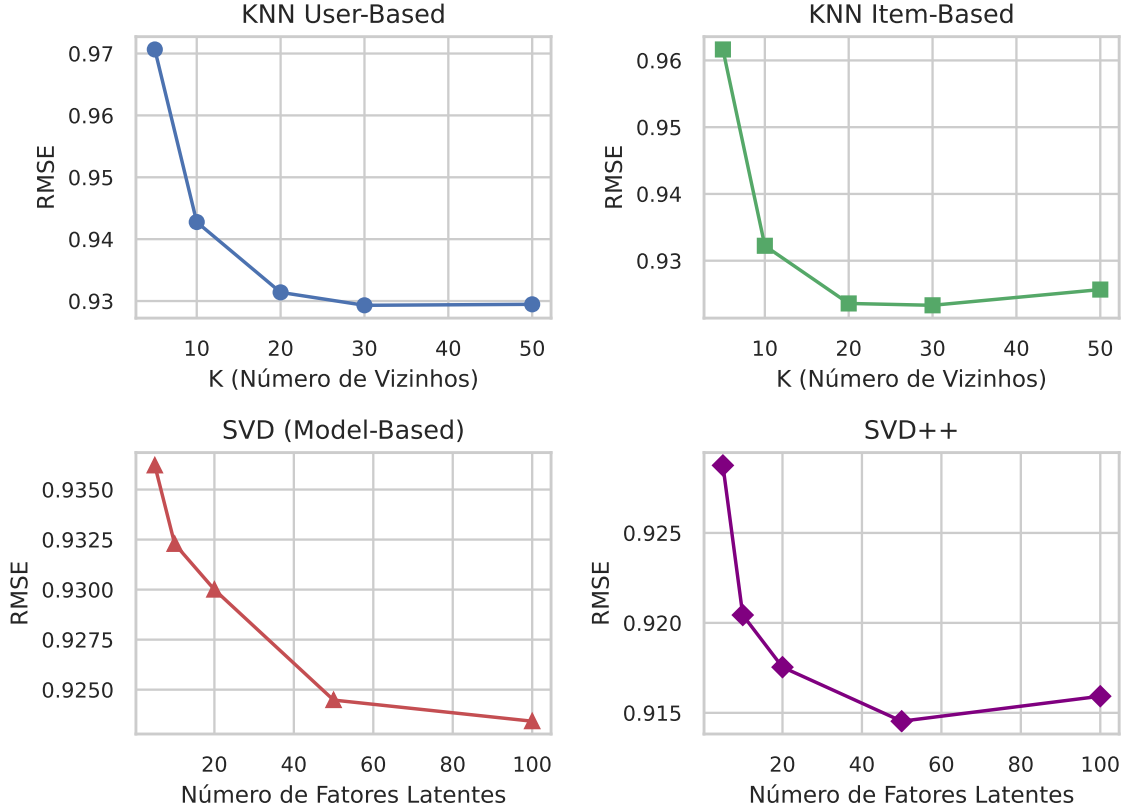


Figura 2: Impacto da variação dos hiperparâmetros na métrica RMSE (Análise via 3-Fold): influência de K nos modelos de vizinhança (KNN) e da dimensão f nos modelos de fatores latentes (SVD e SVD++).

2.4.4 SVD++ — Feedback Implícito. O SVD++ [3] estende o SVD incorporando *feedback implícito*: além dos fatores explícitos $\mathbf{p}_u$, o modelo aprende vetores $\mathbf{y}_j \in \mathbb{R}^f$ para cada item j e usa o conjunto $N(u)$ de itens avaliados pelo usuário (independente da nota) para enriquecer sua representação. A predição passa a ser:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{q}_i^T \left(\mathbf{p}_u + |N(u)|^{-\frac{1}{2}} \sum_{j \in N(u)} \mathbf{y}_j \right) \quad (8)$$

A informação implícita captura o fato de que *quais filmes um usuário se importou em assistir e avaliar* é em si um sinal de preferência, mesmo sem conhecer a nota. Isso permite ao SVD++ construir uma representação mais rica do usuário, especialmente para usuários com poucas avaliações explícitas [3]. O SVD++ foi adicionado ao escopo do trabalho como algoritmo extra com o objetivo de verificar se a incorporação de feedback implícito produz ganho mensurável sobre o SVD padrão no contexto do MovieLens 100k.

3 Resultados e Discussão

A Tabela 5 consolida os resultados finais de todos os modelos, avaliados com K-Fold de 5 folds e os melhores hiperparâmetros encontrados.

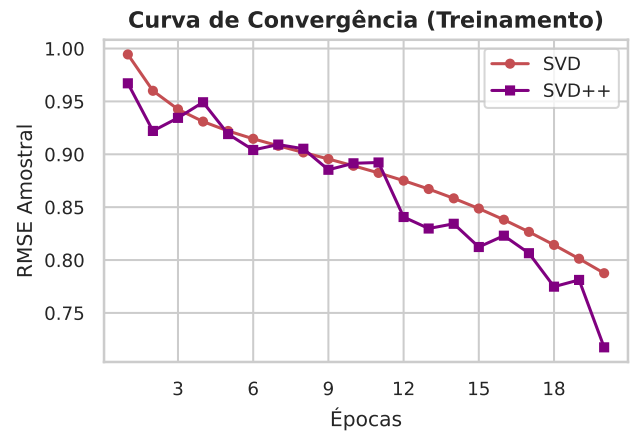


Figura 3: Curvas de convergência do SVD e SVD++ (RMSE de treino por época, média dos 5 folds).

Tabela 5: Resultados Finais — K-Fold ($K = 5$), melhores hiperparâmetros

Modelo	MAE	$\pm$	RMSE	$\pm$
Baseline (Média Global)	0,9447	0,0038	1,1257	0,0046
KNN User-Based ($K = 30$)	0,7280	0,0061	0,9335	0,0065
KNN Item-Based ($K = 30$)	0,7254	0,0046	0,9274	0,0061
SVD ($f = 100$)	0,7311	0,0051	0,9268	0,0054
SVD++ ($f = 50$)	0,7207	0,0046	0,9178	0,0049

Todos os algoritmos implementados superam expressivamente o *baseline* de média global (RMSE = 1,1257), confirmando que os modelos aprenderam padrões significativos nos dados. A redução de RMSE em relação ao *baseline* varia de 17% (KNN User-Based) a 18,5% (SVD++).

Comparação KNN User-Based vs. Item-Based: o KNN Item-Based (RMSE = 0,9274) supera o User-Based (RMSE = 0,9335) em todos os folds, o que é consistente com a literatura [6]. No MovieLens 100k, os filmes recebem em média 59 avaliações, enquanto usuários fazem em média 106 avaliações, mas a distribuição é muito mais assimétrica para usuários (mediana de 65 vs. moda concentrada em valores baixos). Isso torna os perfis de itens mais estáveis para o cálculo de similaridade.

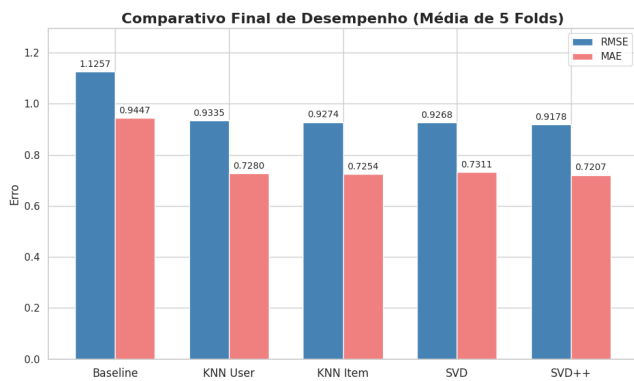


Figura 4: Comparativo final de MAE e RMSE (média de 5 folds) para todos os modelos estudados.

KNN vs. SVD: O SVD ($f = 100$, RMSE = 0,9268) supera o KNN User-Based (RMSE = 0,9335) mas apresenta resultado marginalmente comparável ao KNN Item-Based (RMSE = 0,9274). Isso sugere que, para este dataset de tamanho moderado, a abordagem item-based consegue extrair informação comparável à fatoração de matrizes. A vantagem do SVD sobre o KNN se torna mais pronunciada em datasets maiores e mais esparsos [2].

SVD vs. SVD++: O SVD++ ($f = 50$, RMSE = 0,9178) supera o SVD ($f = 100$, RMSE = 0,9268) com uma redução de 0,009 pontos de RMSE — ganho modesto mas consistente em todos os 5 folds (Figura 5). Isso confirma que a incorporação de *feedback* implícito contribui positivamente para a predição, mesmo no MovieLens 100k onde todos os dados são avaliações explícitas. Vale notar que o melhor número de fatores para o SVD++ ($f = 50$) é menor que para o SVD ($f = 100$): os vetores de *feedback* implícito y_j funcionam como regularização adicional, permitindo que menos fatores explícitos sejam suficientes. Embora o KNN Item-Based apresente um MAE ligeiramente inferior ao SVD, o menor RMSE do SVD indica que a fatoração de matrizes é mais robusta contra erros de predição severos, penalizados quadraticamente pela métrica.

A variação obtida entre os folds foi consideravelmente pequena (desvio padrão $\leq 0,007$), comprovando que a amostragem aleatória estratificada protegeu as partições contra o viés de esparsidade extrema.

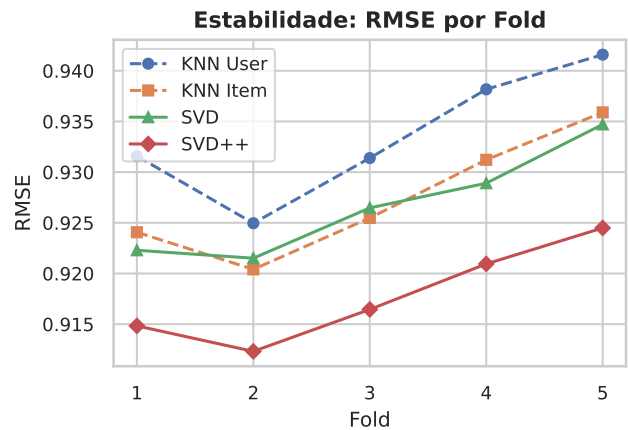


Figura 5: RMSE por fold para todos os modelos otimizados ao longo do processo de validação cruzada externa (5-Fold).

4 Conclusão

Este trabalho implementou e comparou quatro algoritmos de filtragem colaborativa — KNN User-Based, KNN Item-Based, SVD e SVD++ — sobre o dataset MovieLens 100k, utilizando validação cruzada K-Fold com $K = 5$ e as métricas MAE e RMSE. Os principais achados são:

- (1) Todos os algoritmos superam expressivamente o *baseline* de média global, com reduções de RMSE entre 17% e 18,5%;
- (2) O KNN Item-Based supera o User-Based, corroborando resultados da literatura para datasets de tamanho moderado;
- (3) O SVD apresenta desempenho comparável ao KNN Item-Based, com vantagem leve e desvio padrão menor entre folds;
- (4) O SVD++ obteve o melhor desempenho geral (RMSE = 0,9178), confirmando que o *feedback* implícito contribui mesmo em datasets com apenas avaliações explícitas.

Como trabalhos futuros, sugere-se: (i) avaliar os algoritmos em datasets maiores como MovieLens 1M para verificar a escalabilidade; (ii) investigar técnicas de pré-processamento como remoção de outliers e normalização; e (iii) explorar métricas de qualidade de recomendação como Precision@K e Recall@K, complementando a avaliação de predição de notas.

Referências

- [1] G. Adomavicius and A. Tuzhilin. Toward the next generation of recommender systems: a survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, 17(6):734–749, 2005.
- [2] Y. Koren, R. Bell, and C. Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.
- [3] Y. Koren. Factorization meets the neighborhood: a multifaceted collaborative filtering model. In *Proceedings of the 14th ACM SIGKDD*, pp. 426–434, 2008.
- [4] F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor. *Recommender Systems Handbook*. Springer, 2011.
- [5] F. M. Harper and J. A. Konstan. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):1–19, 2015.
- [6] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th WWW Conference*, pp. 285–295, 2001.
- [7] J. Herlocker, J. Konstan, A. Borchers, and J. Riedl. An algorithmic framework for performing collaborative filtering. In *Proceedings of the 22nd SIGIR Conference*, pp. 230–237, 1999.